

UNIVERSIDAD NACIONAL DE COLOMBIA · SEDE BOGOTÁ

Facultad de Ingeniería · Ingeniería de Software 1 (2016701)

biUNestar

Entrega 07 — Presentación final del proyecto

Bienestar universitario: sueño, hidratación, actividad física y estado de ánimo en un solo lugar.

Daniel Felipe Agudelo · Omar Andrés Quiñones · Jaime Javier Andrés Quiñones · Pedro Castiblanco

Departamento de Sistemas e Industrial

Qué vamos a mostrar hoy

Máximo 15 minutos, a contra reloj — el video demostrativo ya está cargado en la carpeta de Drive del curso.

1

Presentación del software

Flujo de uso, funcionalidades principales y secundarias, y cumplimiento de los requerimientos del enunciado inicial.

2

Aspectos técnicos

Arquitectura por capas del proyecto final, decisiones de diseño y los mayores desafíos de aprendizaje.

3

Reflexión final

Aprendizajes del equipo, temas que profundizaríamos y limitaciones que enfrentamos en el proceso.

¿Qué es biUNestar?

Una aplicación para que los estudiantes de la Universidad Nacional registren y monitoreen sus hábitos de sueño, hidratación, actividad física y estado de ánimo — con gráficos, recordatorios y reportes semanales que fomentan el autocuidado durante la vida universitaria.



Sueño

Horas dormidas (0–24)



Hidratación

Vasos o botellas de agua



Actividad física

Minutos de ejercicio



Estado de ánimo

8 emociones + comentario

Acceso exclusivo con correo institucional @unal.edu.co, mediante inicio de sesión con Google — sin manejo de contraseñas propias.

Flujo general de uso



Regla clave: un único registro por día. Si el estudiante lo edita, el sistema sobrescribe el registro existente — nunca crea duplicados.

Funcionalidades principales implementadas



Autenticación con Google

Sign-In institucional (unal.edu.co) + JWT propio, sin contraseñas.



Registro diario de hábitos

Sueño, agua, actividad y ánimo (8 estados + comentario ≤ 500 car.).



Dashboard de resumen

Resumen del día y accesos rápidos a progreso y recordatorios.



Promedios semanales

Cálculo automático de sueño, agua y actividad por semana.



Reportes con gráficas

Evolución semanal de cada hábito en barras/líneas, persistida.



Recordatorios personalizados

CRUD con hora y mensaje propio (ej. "Tomar agua", 10:00 a.m.).

Funcionalidades secundarias y administración



Retroalimentación

Categoría (Sugerencia / Error / Otro) + comentario ≤ 1000 car. Máximo una por día por estudiante.



Recursos de apoyo

El administrador crea, edita y elimina artículos o enlaces de bienestar; los estudiantes los consultan.



Gestión de usuarios y roles

El administrador activa, desactiva y edita datos no sensibles de los perfiles de estudiantes.



Recuperación de cuenta

Protocolo de verificación mediante correo alternativo si se pierde el acceso institucional.

Cumplimiento de los requerimientos del enunciado

	Estado	
Autenticación y roles (RF01–05, RF17)	Implementado	<i>Google Sign-In institucional + JWT + roles Estudiante/Admin</i>
Hábitos y registro diario (RF06–09)	Implementado	<i>Un registro por día, se sobrescribe sin duplicar</i>
Reportes semanales (RF11)	Implementado	<i>Cálculo y persistencia de promedios por semana ISO</i>
Recordatorios (RF12)	Implementado	<i>CRUD propio, verificación de dueño (404 si no aplica)</i>
Recursos de apoyo (RF16)	Implementado	<i>CRUD para admin, lectura para cualquier autenticado</i>
Retroalimentación (RF18)	Implementado	<i>Categoría + límite de una por día, control 409</i>
Exportar reportes en PDF	Pendiente	<i>Could-have del análisis MoSCoW</i>
Pruebas unitarias/integración/sistema (RNF10)	Pendiente	<i>Suite formal con pytest</i>
UI: sliders y botones grandes (RNF07–09)	En frontend	<i>Cubierto en la capa de presentación</i>

Arquitectura del proyecto final

Modelo cliente-servidor: front-end y back-end (FastAPI) desacoplados, comunicados por API REST.

Presentación

Front-end: plantillas / interfaz de usuario. Consume la API vía HTTP.



Lógica de negocio

FastAPI — por módulo: router → service → schema (Pydantic). Reglas del dominio.



Persistencia

SQLAlchemy ORM sobre PostgreSQL, esquema "bienestar" (usuario, hábito, registro_diario, reporte...).

9 módulos con la misma estructura: auth · user · role · habit · daily_record · report · reminder · resource · feedback (model · schema · repository · service · router)

Desafíos técnicos y decisiones clave

¿Qué costó más en el autoaprendizaje de lenguajes y herramientas?

1

Migrar de Django a FastAPI a mitad de proyecto

Reaprender la persistencia y el enrutamiento con un framework nuevo, manteniendo los requerimientos ya analizados en MoSCoW.

2

Autenticación sin contraseña

Verificar el `id_token` de Google Identity Services contra el dominio institucional y emitir un JWT propio, sin custodiar credenciales.

3

Seguridad por diseño

Sin auto-promoción a administrador (se hace manualmente en BD); operaciones sobre recursos ajenos devuelven 404 en vez de 403 para no filtrar información.

4

Bugs heredados del esquema original, ya corregidos

El campo "comentario" estaba limitado a 255 caracteres (debía ser 500); faltaba "categoría" en retroalimentación; no existía CORS para un front-end en otro origen.

Qué falta y qué mejoraríamos a futuro

PENDIENTE DEL ALCANCE ACTUAL

● Exportar reportes en PDF

Requerimiento could-have aún no implementado.

● Suite de pruebas formal

Unitarias, integración, sistema y usabilidad con pytest (RNF10).

● Pulido de interfaz

Sliders para sueño/agua y botones grandes de ánimo (RNF07–09), en curso en el front-end.

IDEAS PARA FUTURAS EDICIONES

● Notificaciones reales

Pasar de recordatorios almacenados a notificaciones push/email efectivas.

● Gamificación de hábitos

Rachas, logros o metas para reforzar la constancia.

● App móvil nativa

Llevar la experiencia más allá del navegador.

Aprendizajes y temas para profundizar

Punto de partida para la reflexión — personalizar con lo que el equipo sienta realmente antes de presentar.

Lo que aprendimos

- Trabajar en equipo de forma distribuida y sostener decisiones grupales bajo tiempo limitado.
- Tomar y asumir una decisión difícil: migrar el back-end de Django a FastAPI a mitad de camino.
- Diseñar autenticación real (OAuth de Google) en lugar de un login de práctica.
- Traducir un documento de requerimientos (MoSCoW) en módulos de código concretos.

Temas que profundizaríamos

- Pruebas automatizadas (unitarias, integración, sistema) desde etapas tempranas del curso.
- Despliegue y DevOps: llevar un proyecto de entorno local a producción real.
- Buenas prácticas de diseño de APIs REST y versionamiento de contratos.
- Más tiempo de retroalimentación entre entregas intermedias.

Limitaciones y dificultades del proceso

Punto de partida para la reflexión — ajustar con la experiencia honesta del equipo, con respeto.

Curva de aprendizaje del nuevo stack

Aprender FastAPI, SQLAlchemy y OAuth de Google desde cero consumió tiempo que se restó a otras entregas.

Coordinación front-end / back-end en paralelo

Front-end y back-end avanzaron con ritmos distintos, lo que exigió sincronizar contratos de API varias veces.

Alcance ambicioso frente al tiempo del curso

El número de requerimientos (18 RF + 10 RNF) fue amplio para el tiempo disponible entre entregas.

Pruebas y documentación quedaron al final

Priorizamos funcionalidad sobre pruebas formales, que quedaron pendientes para el cierre del proyecto.

¡Gracias!

Preguntas y retroalimentación son bienvenidas.